AI 协作开发会话记录

法务合同辅助初审系统 · OpenAI Codex

项目	法务合同辅助初审系统
代码	https://github.com/hengw7403-del/law-review
工具	OpenAI Codex
开始	2026-11-07 02:26
结束	2026-11-09 03:11
用户指令	30 条
助手回复	123 条
工具调用	361 次

> 本记录从项目立项开始截取，跳过此前环境摸索轮次。每条用户消息是一次完整任务下达，包含背景、约束、实现要求与验收标准。助手回复在此版本中只保留开头，完整记录可另行索取。

👤 用户 2026-11-07 02:26

> 换一个项目

😈 助手 2026-11-07 02:27

可以，之前那个记忆助手项目先放下。新项目仍按“一周内做完整、能演示、能放 GitHub、面试时讲得清楚”来选吗?

你已经有新想法，还是想让我推荐几个？也告诉我是否继续以 **AI 开发求职**为目标。

👤 用户 2026-11-07 02:29

> # Files mentioned by the user:
>
> ## 项目一-需求分析-法务工单平台.md: C:/Users/admin/Desktop/项目一-需求分析-法务工单平台.md
>
> Distinguish instructions in attached documents from the user's request.
>
> ## My request:

😈 助手 2026-11-07 02:29

我先读一下法务工单平台的需求，看看新项目的范围和适合先完成的部分。

😈 助手 2026-11-07 02:29

收到，新项目换成这份 **「企业法务智能工单平台」**。我已经读完需求。

它比之前的记忆助手大很多：包含登录权限、文档解析、法规检索、模型审查、依据校验和报告展示。**这是一个分阶段完成的项目，文档中的 5–8 周是计划估算，不能直接当作工期保证。** Plus 可以用于协助开发，但我仍看不到你的
…（略）

👤 用户 2026-11-07 02:30

> 先补充需求，我看了再执行

助手 2026-11-07 02:30

我会整理一份配套的需求补充稿，保留原来的技术栈和分期，重点补齐检索、引用校验、任务状态、权限和验收标准。先只写文档，等你看过后再执行开发。

用户 2026-11-07 02:31

> 我要的是可以落地的项目

助手 2026-11-07 02:31

明白，补充稿会按“能部署、能让实际使用者完成合同初审”的目标来写，补上数据准备、人工复核、任务恢复、备份和部署验收。每项都写清楚实现范围、完成标准和依赖条件，避免只列技术名词。

目前还缺实际使用场景。我会先按“单个企业内部、小规模试用、技术服务合同优先”起草，并把这些假设标出来，供你审阅。

助手 2026-11-07 02:34

使用场景明确了：3 人、技术服务合同、内网 Docker。

我会据此收敛设计：保留单企业和两级角色，增加可恢复的后台任务、人工确认流程，以及容器重启后数据和报告不丢失的验收。另外会明确一点：内网部署后，调用 DeepSeek 仍会把选定文本发送到外部 API，这需要在接入真实合同前确定。

助手 2026-11-07 02:37

已按 **3 人使用 / 技术服务合同 / 内网 Docker** 写好落地需求补充稿：

[查看《法务工单平台—落地需求 v4 审阅稿》](C:/Users/admin/Documents/Codex/2026-11-03/ni-sh/outputs/法务工单平台-落地需求-v4-审阅稿.md)

重点补
…（略）

用户 2026-11-07 02:42

> # Files mentioned by the user:
>
> ## 项目一-需求分析-v5.md: C:/Users/admin/Desktop/项目一-需求分析-v5.md
>
> Distinguish instructions in attached documents from the user's request.
>
> ## My request:

助手 2026-11-07 02:42

我先读一下 v5，对照上一版检查需求变化和落地条件；这一步只审阅，不开始开发。

助手 2026-11-07 02:42

看完了。**v5 比 v4 更适合一个人实施：七张表、单 worker、先用 BM25，这些收敛可以保留。** 不需要把删掉的复杂功能重新加回来。

但目前还不能直接作为实施基线，有六处需要补齐，否则开发时会卡住或验收对不上：

1. **报告确认没有地方存。**
   文中要求 `confir
…（略）

用户 2026-11-07 02:43

> 写出v6的版本

助手 2026-11-07 02:43

我会把 v5 整理成独立、可直接审阅的 v6，保留七张表和单 worker 的精简方案，补齐报告冻结、人工复核、任务去重、检索校准及部署验收规则。本次只写需求文档，不开始开发。

助手 2026-11-07 02:48

v6 正文已写好。七张表的范围保持不变，报告确认数据放在任务表中；机器原始结果、人工修改和历史记录也有了明确存储位置。我正在检查 SQL 是否可执行，以及去重和报告确认约束是否有效。

助手 2026-11-07 02:48

[v6 完整需求文档已写好，点击查看](C:/Users/admin/Documents/Codex/2026-11-03/ni-sh/outputs/项目一-需求分析-v6.md)。

保留了 **七张表、单 worker、BM25 首版**，补齐人工复核、报告冻结、任务恢复、权限和部署验收规则。
…（略）

用户 2026-11-07 02:51

项目：法务合同辅助初审系统
位置：C:\Users\admin\Documents\deepseek-harness\default-workspace\law-review\
环境：Windows + Python 3.9.6

【重要】Python 3.9 不支持 X | None 语法，类型注解必须用 Optional[X] / List[str]，
不能用 3.10+ 的写法。

本轮任务：只搭骨架 + 建表，不写业务逻辑。

1. 目录结构
app/main.py FastAPI 入口
app/db.py SQLAlchemy 引擎与会话
app/models.py 7 张表
app/api/ 路由（先空）
tests/

2. requirements.txt
fastapi, uvicorn[standard], sqlalchemy, alembic, pydantic,
python-multipart, pdfplumber, python-docx,
jieba, rank-bm25, python-jose[cryptography], bcrypt

3. 数据库：SQLite，7 张表
users / contracts / clauses / knowledge_chunks
/ review_tasks / review_items / llm_logs
（表结构以需求文档第 9 节为准，注意非空 item_key 的唯一约束）

4. 两个健康检查
GET /health/live → 200 {"status":"ok"}
GET /health/ready → 检查数据库可读写

约束：
- 每完成一步先跑通再继续
- 不要写审查、检索、模型调用等业务代码
- 告诉我实际安装了哪些版本

验收：
- pip install -r requirements.txt 成功
- uvicorn app.main:app 能起来
- /health/live 返回 200
- law.db 生成，7 张表都在

助手 2026-11-07 02:51

我先检查目标目录、Python 版本和 v6 第 9 节表结构，再依次完成目录骨架、依赖安装、建表和健康检查。每一步验证通过后再继续，本轮不写审查、检索或模型调用代码。

助手 2026-11-07 02:52

目标目录原先不存在，已创建骨架，并确认虚拟环境使用 Python 3.9.6。依赖安装遇到了旧版 pip 和当前代理配置的问题；我会先修复安装链路，安装成功后再继续建表。

助手 2026-11-07 02:54

已确认 Python 3.9.6 自带的 pip 是 21.1.3，目标环境也存在系统代理配置。我正在单独更新项目虚拟环境里的 pip，不改系统 Python；依赖安装仍是当前这一步的阻塞点。

助手 2026-11-07 02:56

pip 已更新到 25.3，依赖现在能正常下载，并在按 Python 3.9 兼容性解析版本。安装完成后，我会记录实际版本，再逐项核对 v6 的字段、外键、检查约束和索引，重点验证空 `clause_id` 的检查项也不能重复写入。

助手 2026-11-07 02:58

自动选择依赖版本时解析耗时较长。我会改用明确支持 Python 3.9 的固定版本安装，再以实际安装和运行结果验证；最终会同时保存直接依赖版本和完整依赖锁定清单。

助手 2026-11-07 03:00

安装包下载已恢复正常，但部分间接依赖仍在耗时解析版本。我会一并固定这些依赖，并优先使用 Windows 预编译包，避免引入额外编译工具。骨架和数据库这一步会继续完成。

助手 2026-11-07 03:02

安装仍卡在在线索引解析。我已改为下载与 Python 3.9/Windows 匹配的固定安装包，并校验包哈希，再从本地安装；这不会修改系统代理或系统 Python。依赖就绪后再继续建表验证。

助手 2026-11-07 03:04

依赖已安装完成，`pip check` 通过，Python 仍是 3.9.6。现在开始建表：按 v6 第 9 节生成七个 SQLAlchemy 模型，并核对字段、外键、唯一约束和部分唯一索引；验证通过后再加健康接口。

助手 2026-11-07 03:07

七张表已生成，数据库结构检查通过：字段、默认值、外键和索引与 v6 一致。测试也确认了非空 `item_key` 去重、同一合同仅一个活动任务，以及数据库只读时就绪检查失败的行为。接下来加两个健康接口，并实际启动 Uvicorn 验收。

助手 2026-11-07 03:10

已完成，项目位于：
`C:\Users\admin\Documents\deepseek-harness\default-workspace\law-review\`

本轮仅实现骨架、七张表和健康检查，使用 **Python 3.9.6**，没有使用 `X | None` 注解。

验收通过：
…（略）

用户 2026-11-07 03:15

接上一轮的骨架，本轮做【文档解析 + 条款切分】，仍然不接模型。

【目标】
上传一份合同文件，返回切分好的条款列表。先不存库，做无状态预览接口，
等登录做完再接正式流程。

【要实现的模块】

1. app/services/parser.py —— 文档解析
   - PDF: pdfplumber，逐页提取文本
   - DOCX: python-docx，按段落提取
   - TXT: UTF-8 读取，编码错误要捕获
   - 统一返回 (text, page_map)：page_map 记录每段文本对应第几页/第几段

2. app/services/clause_splitter.py —— 条款切分
   - 主规则：匹配行首的「第X条」「第X条之一」，X 支持中文数字和阿拉伯数字
   - 次规则：匹配"一、""1.""1.1""（一）」等多级编号
   - 回退：以上都不命中时，按空行分段
   - 每条返回：seq(序号)、original_no(原编号如"第七条")、content(原文)、
     char_start/char_end(字符偏移)、location(页码或段落号)

3. app/services/parse_quality.py —— 解析质量判定
   - 文本 < 50 字符 → 失败，error=EMPTY_TEXT
   - 替换字符(\\ufffd)占比 > 5% → 失败，error=GARBLED
   - 切出条款 < 3 → 成功但 warning=TOO_FEW_CLAUSES
   - 返回 (ok: bool, error: str|None, warnings: list[str])

4. app/api/parse.py —— 预览接口
   POST /parse/preview
   入参: multipart file
   出参: {
     "filename": "...",
     "text_length": 12345,
     "clause_count": 24,
     "warnings": [],
     "clauses": [
       {"seq":1,"original_no":"第一条","content":"...","char_start":0,
        "char_end":120,"location":"p1"}
     ]
   }

【约束】
- 单文件上限 20MB，超过返回 413
- 扩展名和实际格式要校验（PDF 看文件头 %PDF，DOCX 看是不是 ZIP）
- 三种失败要有明确错误码：EMPTY_TEXT / GARBLED / UNSUPPORTED_FORMAT
- 解析放独立函数，不要写进路由里

【顺便做】
在 samples/ 目录生成 2 份测试用的技术服务合同（Markdown 或 txt）：
- 合同A：条款规范，用「第X条」编号，含 10-12 条
- 合同B：编号混乱（混用"一、"和"1."），含跨段落的长条款
内容要像真的技术服务合同（服务范围、价款、交付、验收、违约、
知识产权、保密、争议解决都要有）。

> 【验收】
> 1. 上传合同A，切出 10-12 条，编号正确
> 2. 上传合同B，能按回退规则切出合理条款
> 3. 上传一个空 txt，返回 EMPTY_TEXT
> 4. 上传一个纯图片 PDF，返回 EMPTY_TEXT 或 GARBLED（不是 500）
> 5. 上传 .exe 改名成 .pdf，返回 UNSUPPORTED_FORMAT
> 6. 每份合同的切分结果打印条款数和前 3 条内容供人工核对

助手 2026-11-07 03:15

我先检查现有骨架和依赖，再依次实现解析、条款切分、质量判定和预览接口。保持 Python 3.9.6 兼容；本轮只做无状态预览，不写数据库、不接模型，并用两份样本及异常文件逐项验收。

助手 2026-11-07 03:17

现有骨架和依赖可直接复用。切分会按“第 X 条”优先、混合编号其次、空行最后处理；存在上级条款时，内部的子编号和跨段落正文保留在该条款内。字符偏移以解析后返回的文本为准，并验证 `content == text[char_start:char_end]`。

助手 2026-11-07 03:25

合同 A 已切出 12 条，合同 B 切出 10 条，跨段落的长条款和内部子编号均保留。接口测试发现，pdfplumber 会将损坏 PDF 的异常包装成另一种异常，导致这类文件返回 500；我正在补齐捕获，并重新跑异常上传和原有骨架测试。

助手 2026-11-07 03:27

26 项测试已全部通过，包括 PDF/DOCX/TXT 上传、跨页定位、空行回退、纯图片 PDF、伪装 PDF、错误编码和 20 MB 限制。预览请求未写入业务表。正在生成两份合同的条款数、前 3 条完整内容及验收记录，供你人工核对。

助手 2026-11-07 03:29

已完成 **文档解析、条款切分和 `POST /parse/preview`**，兼容 Python 3.9.6，无预览数据入库、无模型调用。

26 项测试全部通过：

验收项	结果
合同 A	12 条，编号正确
合同 B	10 条，跨段长条款及
…（略）

用户 2026-11-07 03:31

接上一轮。本轮做【知识库入库 + BM25 检索】，仍然不接模型。

【要实现的模块】

1. app/services/chunker.py —— 法规切块
   - 输入：一部法规的全文 + 文件名 + 来源信息
   - 切块规则：优先按「第X条」切，每条一块；无编号时按段落切（每块 ≤ 500 字）
   - 每块保留：article_no(条号)、chunk_index、content、content_hash(sha256)

2. app/services/knowledge.py —— 入库
   - 把切好的块写入 knowledge_chunks 表
   - 同一 source_id + source_version 重复导入要幂等（先删后插或跳过已存在）
   - 默认 verified=0、enabled=0（草稿状态），需要显式启用才能被检索

3. app/services/search.py —— BM25 检索
   - jieba 分词，查询和文档用同一套分词规则
   - rank_bm25 的 BM25Okapi
   - 只检索 enabled=1 且 verified=1 且 superseded=0 的块
   - 返回 Top-K（默认 5），每项含：
     {chunk_id, source_file, article_no, content, score, matched_terms}

4. app/api/search.py —— 接口
   POST /search
   入参 {query: str, top_k: int = 5}
   出参 {query, results: [...]}

5. app/cli.py 或 scripts/ —— 导入命令
   python -m app.cli import-law --file <路径> --source-id <标识> --name <法规名>
   python -m app.cli enable-source --source-id <标识>　（把该来源所有块设为 verified=1, enabled=1）

【样本法规】
在 samples/laws/ 下准备 2 部法规的关键条文（Markdown 或 txt）：
- 民法典合同编：违约责任相关（第 577-594 条附近），约 15 条
- 民法典合同编：合同解除与争议解决（第 563-566、507 条附近），约 8 条

【重要】只写你确信的条文原文，不确定的不要编。
每部文件开头注明「测试用，条文需人工核对」。

【约束】
- 阈值不写死。BM25 分数不是 0-1 概率，本轮只返回原始分数 + 命中的词，
  由调用方判断，阈值等标注集出来再校准
- 索引在启动时构建，数据量小（几十条），不做增量
- 分词要保留法条编号（「第五百八十五条」不能被切碎）

【验收】
1. 导入两部法规，knowledge_chunks 里有记录，enabled=0
2. 未启用时检索「违约金」返回空
3. 启用后检索「违约金过高怎么办」→ 第一条命中民法典第 585 条
4. 检索「合同怎么解除」→ 命中第 563 条
5. 检索「今天天气怎么样」→ 分数明显低于前两个查询（打印对比）
6. 返回的 matched_terms 能看到哪些词命中了

助手 2026-11-07 03:31

我先核对现有表结构和接口，再从官方来源核实样本条文原文。随后按“切块与草稿入库 → 显式启用 → 启动构建 BM25 索引 → 检索验收”逐步实现，本轮不接模型，也不设置分数阈值。

助手 2026-11-07 03:34

样本原文已从最高人民法院公布的《民法典》页面取得。我会准备两个节选文件：违约责任第 577–591 条共 15 条，以及解除、终止和争议条款相关的 8 条；文件开头保留“测试用，条文需人工核对”，导入时默认仍为草稿。

另外，索引只在服务启动时全量构建。CLI 导入或启用之后，需要重启服务才能检索到
…（略）

助手 2026-11-07 03:39

草稿入库和重复导入检查已通过，共 23 个块。检索测试发现，jieba 普通分词会把“解除合同”合成一个词，漏掉查询中的“合同”和“解除”。已改用查询、文档共用的搜索模式，并保留法条编号和“过高”等完整词；目前第 585 条排首位，第 563 条进入解除查询的 Top-5，未按条号额外加权。

助手 2026-11-07 03:42

38 项测试已全部通过，包括幂等导入、草稿不可检索、三个状态字段的过滤、启动快照、停用后立即排除，以及零分和负分仍按原值返回。

接下来会对项目的 `law.db` 执行你要求的导入和显式启用验收，保留 23 个已启用的测试块，并打印三个查询的原始分数和命中词对比。

助手 2026-11-07 03:44

已完成知识库切块、幂等入库、启用 CLI 和 `POST /search`，仍未接模型。**38 项测试全部通过。**

实际验收：

查询	结果	原始最高分
违约金过高怎么办	第 585 条排第一；命中“违约、违约金”	3.851218
…（略）

用户 2026-11-07 03:46

接上一轮。本轮接入 LLM，做【单条款风险判定 + 依据校验】。
先只做单条款的端到端，不做整份合同的编排。

【环境】
DeepSeek API（OpenAI 兼容），key 从环境变量 DEEPSEEK_API_KEY 读。
base_url: https://api.deepseek.com
model: deepseek-chat
temperature: 0

【要实现的模块】

1. app/services/llm.py —— 模型客户端
   - 用 openai SDK（兼容接口）
   - 一次调用返回 (content, usage, duration_ms)
   - 记录 prompt_tokens / completion_tokens 到 llm_logs 表
   - 超时 60 秒，最多重试 2 次（指数退避）
   - 认证错误、4xx 不重试，直接抛

2. app/prompts/risk_judge.py —— 提示词模板
   输入：条款原文、我方立场、检索到的候选依据（带 ID）
   要求模型输出严格 JSON：
   ```
   {
     "has_issue": true/false,
     "risk_level": "high"/"medium"/"low"/"none",
     "description": "问题描述",
     "reason": "判断理由",
     "suggestion": "修改建议",
     "evidence_ids": ["k_12", "k_45"],
     "support_note": "每条依据支持哪个结论"
   }
   ```

   提示词里必须强调：
   - evidence_ids 只能从给定的候选 ID 里选，不能编
   - 没有合适依据时 evidence_ids 留空，不要硬凑
   - 站在「我方立场」判断（同样的条款，甲方乙方风险不同）

3. app/services/validator.py —— 四层校验
   第一层：evidence_ids 必须在本次候选集里，不在的丢弃
   第二层：依据的原文、条号、来源由服务端从 DB 填充，不用模型写的
   第三层：has_issue=true 但 evidence 为空 → 降级 insufficient，risk_level=unknown
   第四层：JSON 解析失败或字段缺失 → 重试一次，仍失败标 failed

4. app/services/risk_judge.py —— 编排
   judge_clause(clause_text, stance, rule_id) -> result
   - 先调检索拿 Top-5
   - 组 prompt
   - 调模型
   - 过校验
   - 返回 {check_status, risk_level, description, suggestion, evidence[], usage}

5. app/api/judge.py —— 预览接口
   POST /judge/preview
   入参 {clause_text: str, stance: "party_a"|"party_b", rule_id: str}

出参 校验后的完整结果

【约束】
- 不要用 Function Calling，用确定性流程
- 模型不能自己决定调什么工具
- 提示词模板单独成文件，方便后面对比版本
- temperature=0

【验收】
1. 用合同 A 的「违约责任」条款（违约金未设上限），
stance=party_a，rule_id=liability
期望：risk_level 为 high 或 medium，description 提到"上限"或"过高"

2. 用一条知识库里完全没有的条款（比如"数据出境"），
期望：check_status=insufficient，risk_level=unknown，不是硬编结论

3. 故意构造一个模型返回假 evidence_id 的用例（可用 mock 或临时改提示词），
期望：假 ID 被丢弃，若因此无依据则降级 insufficient

4. 打印每次调用的 token 消耗和耗时

【成本提示】
所有测试加起来控制在 20 次调用以内。

助手 2026-11-07 03:46

我先检查当前检索、日志表和 DeepSeek 配置，再接入单条款的确定性判定流程。会先用 mock 验证重试和假依据处理，真实 API 调用另行计数，确保本轮不超过 20 次；不会打印 API key。

助手 2026-11-07 03:48

有两处需要同步处理：现有合同 A 的违约责任条款已经写了“累计不超过百分之十”，我会另建“去掉上限”的测试变体，不改原样本，也不强迫模型对原文报高风险。另一个是预览没有正式审查任务，日志会按 `request_id` 关联，并让 `llm_logs.review_id` 可空，避免伪造合同、用户或任
…（略）

助手 2026-11-07 03:54

SDK 和单条款流程已接上，超时/服务端错误重试与 JSON 格式修复分开计数，SDK 自带重试已关闭，避免调用次数翻倍。正在用本地 mock 验证：4xx 不重试、假 ID 丢弃、无依据降级，以及格式修复失败后返回 `failed`。目前真实 DeepSeek 调用为 0 次，环境里仍未读到 ke
…（略）

助手 2026-11-07 04:01

收到。我会只加载项目根目录的 `.env`，且不覆盖已有环境变量；确认 `.env` 已被 Git 忽略，并将 `python-dotenv` 列为直接依赖。mock 测试会隔离真实 key，随后只运行约定的两项真实验收，重试也计入 20 次预算。

助手 2026-11-07 04:03

已确认项目根目录 `.env` 中的 key 可读取，`.gitignore` 也已包含 `.env`，未输出密钥值。58 项离线测试和依赖检查全部通过，现在开始真实 DeepSeek 验收；假依据用例仍使用 mock。

助手 2026-11-07 04:04

真实请求遇到连接错误，已按配置尝试 3 次并停止，尚未拿到模型回答，token 用量也未返回。我先做不带 key 的连通性检查，定位代理或证书问题；这 3 次会保留在验收记录中，后续最多再允许 17 次真实尝试。

助手 2026-11-07 04:06

真实验收已通过：无上限变体返回 `medium` 并引用第 585 条；数据出境条款返回 `insufficient / unknown`。累计真实尝试为 5 次，其中 3 次是此前的连接失败，成功回答 2 次，低于 20 次上限。

成功请求的 token 和耗时已落库。请求参数仍是 `deeps
…（略）

助手 2026-11-07 04:08

已加入 `python-dotenv==1.1.0`，自动读取项目根目录 `.env`，已有环境变量优先。确认 `.gitignore` 已包含 `.env`，密钥未输出。

单条款流程已完成，**58 项测试通过**，真实验收结果：

| 用例 | 结果 | 输入/输出 token | 耗时 |
…（略）

用户 2026-11-07 04:12

接上一轮。本轮把单条款判定串成完整任务：建任务 → worker 逐条执行 → 落库 → 出报告。

【重要：先限流测试】
测试阶段只跑前 5 条条款、只用 3 个规则，控制成本在 20 次模型调用以内。
跑通后再放开全量。

【要实现的模块】

1. app/services/review_planner.py —— 生成检查计划
   - 输入：合同的所有父条款 + 规则清单
   - 输出：检查单元列表
   - 每个单元：item_key、clause_id、rule_id
   - item_key 格式：
     条款级：clause:<clause_id>:<rule_id>
     合同级（缺失检查，本轮先不做）：contract:<rule_id>
   - 本轮只用 3 个规则：liability（违约责任）、payment（付款条件）、
     dispute_resolution（争议解决）

2. app/services/review_worker.py —— worker
   - 独立进程，轮询 review_tasks 表领任务
   - 领取：SELECT 最早 queued → UPDATE 成 running（短事务）
   - 拿到任务后：预建 review_items（全部 pending），
     然后逐单元执行 judge_clause，写结果，更新 done_items
   - 每完成一个单元就提交事务（不能攒到最后）
   - 单单元失败：写 failed 结果，继续下一个
   - 全部完成：有失败 → partial_failed，无失败 → completed
   - 启动时把遗留的 running 任务重置为 queued

3. app/services/report.py —— 报告生成
   - 从 review_items 生成 Markdown
   - 结构：合同信息 / 统计（高·中·低·依据不足·失败）/
     逐项意见（条款原文 + 风险 + 依据）/ 未发现问题清单
   - 缺失项（clause_id 为空）单独列一节

4. app/cli.py 增加命令
   python -m app.cli run-worker   启动 worker
   python -m app.cli review --contract <id> --max-clauses 5 --rules liability,payment

5. 接口
   POST /contracts/{id}/review → 202 {task_id}
   GET /tasks/{id}   → 状态和进度
   GET /reviews/{id}/report   → Markdown

【约束】
- 幂等：item_key 唯一，重复执行不重复插入
- 去重：同一合同已有 queued/running 任务时，返回已有 task_id
- 每次模型调用都记 llm_logs
- 进度要真实（done_items 包含成功的，failed_items 单列）
- worker 不占用数据库写事务做模型调用

【验收】
1. 合同 A 建任务，max_clauses=5，规则=liability
   → 任务从 queued 到 running 到 completed

→ review_items 有 5 条，每条有 machine_result
→ 报告能生成，5 条都在

2. 任务跑到一半 kill worker，重启后能继续
→ 已完成的单元不重复调用（看 llm_logs 的调用次数）

3. 重复提交同一合同
→ 返回相同的 task_id，不新建

4. 打印：模型调用次数、总 token、总耗时

助手 2026-11-07 04:12

我会先检查现有任务表、单条款判定和日志接口，再接任务创建、独立 worker、逐项落库及报告。真实验收先限定合同 A 前 5 条、`liability` 规则；重启恢复和去重先用 mock 验证，所有真实模型尝试（含重试）共用本轮 20 次预算。

助手 2026-11-07 04:15

任务将保存条款、规则和知识库快照，重启后按同一计划续跑。为防止重启绕过费用限制，我会在正式任务发出模型请求前先预记日志，预算计数包含重试和被中断的尝试；已经落库完成的单元会直接跳过。

进度会明确区分：`done_items` 是已完成且未执行失败的数量，`failed_items` 单列，`pro
…（略）

助手 2026-11-07 04:21

任务创建、worker 检查点落库和 Markdown 报告已接通。独立进程 mock 测试已确认：kill 后重启能续跑，已保存单元的调用次数不增加；第二个 worker 会因进程锁退出，不会重置正在执行的任务。接着验证失败隔离、接口和持久预算，再跑受限真实任务。

助手 2026-11-07 04:24

受限任务的 mock 端到端验收已通过：5 个检查单元全部落库，报告包含前 5 条，重复提交返回同一任务 ID。

现在只运行一份真实任务（前 5 条、`liability`）。会在第 2 个单元提交后、下一次调用前 kill worker，再启动续跑，核对已完成单元的日志次数是否保持不变。

助手 2026-11-07 04:28

真实任务已完成，5 个单元均已保存 `machine_result`，报告包含前 5 条。kill/restart 前后，前两个已完成单元的调用记录都是 2 条，没有重复调用。

本轮共发出 5 次模型请求，合计 7,554 token；模型累计耗时 11.807 秒，任务总耗时约 20.861 秒
…（略）

助手 2026-11-07 04:30

已完成任务创建、独立 worker、逐项落库和 Markdown 报告，**67 项测试通过**。

真实验收结果：

- 前 5 条、`liability`：`queued → running → completed`，5 项均有 `machine_result`。
- 完成 2 项后 kill/
…（略）

用户 2026-11-07 04:32

接上一轮。本轮做三件事：规则适用条件、缺失项检查、条款前言剥离。

【一、规则适用条件】—— 省成本的关键

现在每个条款都要跑每个规则，24 条款 × 3 规则 = 72 次调用，
其中大部分是用错的规则查错的条款，纯浪费。

给每个规则加适用条件。规则定义改成：

```
{
  "rule_id": "liability",
  "name": "违约责任",
  "scope": "clause",
  "applicable_when": {
    "keywords": ["违约", "赔偿", "责任", "罚", "逾期", "迟延", "损失"],
    "min_hits": 1
  }
}
```

生成检查计划时：
- 条款命中 keywords 至少 min_hits 个 → 生成条款级检查单元
- 一个都不命中 → 不生成单元，记为 not_applicable（不调模型、不算失败）

三个规则的适用关键词建议：
- liability（违约责任）：违约、赔偿、责任、罚、逾期、迟延、损失
- payment（付款条件）：价款、付款、支付、款项、发票、结算、账期
- dispute_resolution（争议解决）：争议、仲裁、诉讼、法院、管辖、协商

【二、缺失项检查】—— 只跑一次，成本低

有些风险不是"条款有问题"，是"根本没写"。

加一个合同级检查：
- item_key = contract:<rule_id>
- clause_id 为空
- 输入：全合同的条款清单（条号 + 每条前 80 字），不是全文
- 问模型：这些检查项里，哪些合同完全没约定？
- 输出：缺失项列表 + 每项说明为什么重要

先做 3 个缺失项规则：
- 争议解决（没约定去哪打官司）
- 验收标准（没约定怎么算验收通过）
- 保密（没约定保密义务）

合同级检查每个规则只调一次模型，不是每个条款调一次。

【三、条款前言剥离】

现在第一条会把合同标题、甲乙方那段都粘进去（450 字符），影响检索。
切分时把「第一条」之前的内容单独存为前言，不算条款。
前言保留在 contracts.raw_text 里，但 clause 表里不出现。

【四、状态增加 not_applicable】

review_items.check_status 增加一个枚举值：not_applicable
含义：本规则不适用于本条款，未做检查。
不要和 no_issue_detected（检查了但没问题）混。

报告统计表增加「不适用」一列，和「未发现问题」分开。

【验收】
1. 合同 A 前 12 条 × liability 规则
   → 只有含违约责任关键词的条款生成单元（预计 3-5 条）
   → 其余记 not_applicable
   → 打印对比：改前 12 次调用，改后几次

2. 3 个缺失项检查
   → 各调 1 次模型
   → 合同 A 应该在「保密」或「验收」上给出缺失结论（看样本实际情况）
   → 报告里单独一节「缺失项」

3. 合同 A 重新切分
   → 第一条不再包含合同标题和甲乙方
   → 前言单独存放

【成本】
本轮测试控制在 15 次调用以内。

助手 2026-11-07 04:32

我会先核对合同 A、计划生成和状态约束，再加关键词适用过滤、合同级缺失检查和前言剥离。本轮真实调用（含重试）共用 15 次预算。缺失项会按样本实际内容验收，不会为了得到“缺失”结论而忽略合同中已有的约定。

助手 2026-11-07 04:35

样本核对结果有两点需要说明：按你给定的关键词，合同 A 实际只有第十条命中 `liability`，预计可从 12 次降到 1 次；第五条已写验收标准，第八条已写保密义务，不应报成缺失。第十二条有争议解决表述，但具体机构约定较笼统，检查时会区分“完全缺失”和“内容不完整”。

不适用条款不会进入可执
…（略）

助手 2026-11-07 04:43

关键词过滤和前言剥离已通过原有解析、worker 测试。合同级检查会单独处理：每个规则只发出一次请求，失败也不自动重试；输入只含条号、ID 和每条前 80 字。由于摘要不能证明全文没有约定，服务端还会保守检查完整正文中的相关内容，避免将摘要截断误报成缺失。

助手 2026-11-07 04:50

离线验证已跑通：合同 A 的 12 条中，只有 1 条命中违约责任关键词，其余 11 条记为“不适用”；三个合同级检查各执行一次。接下来跑完整测试，再进行真实 API 验收，控制在本轮 15 次调用以内。

助手 2026-11-07 04:52

78 项自动化测试全部通过，真实验收正在运行。合同 A 已有验收和保密约定，不能为了满足预期而报“缺失”；这次会区分“已有约定”“约定不完整”和“摘要不足以判断”。旧合同和历史审查记录保留，剥离前言后的条款存入新版本。

助手 2026-11-07 04:52

真实验收已完成：共 4 次模型调用、6064 tokens，累计模型耗时约 8.3 秒，未超预算。结果是争议解决方式不完整，验收和保密已有约定；违约责任检查因依据不足降为 `insufficient`。任务执行成功不代表合同通过审查，这些结果已分别写入报告。

助手 2026-11-07 04:54

已完成规则适用过滤、合同级缺失检查、前言剥离，以及 `not_applicable` 入库和报告统计。

验收结果：

- 合同 A：条款级调用 **12 → 1 次**，其余 11 条记为不适用。
- 三个合同级检查各调用一次；本轮共 **4/15 次**，6064 tokens，模型耗时约 8.
…（略）

用户 2026-11-07 08:09

> 【要改什么】
> 把「违约责任」规则的关键词表补宽。
>
> 【补哪些词】
> 除了现有的「违约」，还要覆盖这些表达：
> 违反、不履行、未履行、未按期、未按时、延迟、迟延、逾期、超时、赔偿、损失
>
> 【为什么】
> 实测时第十一条「解除与终止」被跳过，原文写的是
> 「一方严重违反保密、付款或交付义务…」，
> 关键词表里只有「违约」没有「违反」，所以没命中。
> 第三条原文用的是「延迟」，词表里是「迟延」，也没命中。
>
> 【怎么算改对】
> - 第十一条应被判定为适用，不再跳过
> - 第三条也应命中
> - 第一、六、七、十二条仍不适用（不含违约内容）
> - 打印改前后的调用次数对比

助手 2026-11-07 08:09

我会补全违约责任关键词，用合同 A 核对第三、十一条及应跳过的条款，并打印修改前后的预计调用数；这次验证不调用模型。

助手 2026-11-07 08:10

计划对比已确认：第三条命中“延迟”，第十一条命中“违反”；第一、六、七、十二条仍不适用。条款级预计调用数从扩词前的 1 次变为 3 次，相比未过滤的 12 次仍少 9 次。相关回归测试正在收尾。

助手 2026-11-07 08:10

已补全关键词，22 项相关测试通过。

- 第三条命中“延迟”，第十一条命中“违反”，均适用。
- 第一、六、七、十二条仍不适用。
- 预计条款级调用：**扩词前 1 次 → 扩词后 3 次**；未过滤时为 12 次。
- 本次实际模型调用 **0 次**，历史任务结果保留。

[查看逐条对比](C
…（略）

用户 2026-11-07 08:12

接上一轮。本轮补全合同级缺失检查。

【背景】
条款级检查只能发现"写了什么但写得不对"，
发现不了"该写什么但根本没写"。
合同 A 的第二条（付款）和第九条（质保）都只约定了义务、
没约定违反的后果，条款级检查会跳过它们，这类风险要靠缺失检查兜。

【要加的两个缺失规则】

1. rule_id: payment_default
   名称：逾期付款违约责任缺失
   检查目标：合同是否约定了甲方逾期付款的违约金或赔偿责任
   为什么重要：没约定时，乙方只能主张实际损失，举证难、周期长

2. rule_id: delivery_default
   名称：交付违约责任缺失
   检查目标：合同是否约定了乙方逾期交付、交付不合格的责任
   为什么重要：没约定时，甲方难以主张违约金，只能另行协商或诉讼

【实现】
- 合同级单元 item_key = contract:<rule_id>，clause_id 为空
- 每个规则调一次模型，输入是全部条款的清单（条号 + 每条前 80 字）
- 提示词要求模型输出：
  {has_gap: bool, gap_description: "缺什么", impact: "缺了会导致什么后果", suggestion: "建议怎么补"}
- has_gap=false 时 check_status=no_issue_detected
- 输出必须有 impact 字段，不能只说"未约定"

【报告】
缺失项单独一节「合同级缺失」，和逐项意见分开。
每项要写清：缺什么 / 缺了会怎样 / 建议补什么。

【验收】
1. 合同 A 跑 payment_default
   → 应该 has_gap=true（第二条只说了"收到发票后付款"，没说逾期怎么办）
   → impact 要具体，不能是"存在风险"这种空话

2. 合同 A 跑 delivery_default
   → 看实际结果（第十条已有违约金，可能不算缺）
   → 如果判定无缺失，要能说清理由（第十条已覆盖）

3. 报告里「合同级缺失」一节正常显示，clause_id 为空的项不伪造条号

【成本】
新增 2 次调用，本轮总计控制在 3 次以内。

助手 2026-11-07 08:12

我会加上这两个合同级规则，补齐“缺什么 / 影响 / 建议”的输出和报告展示，先做 mock 验证，再各调用一次模型。先核对一个关键点：合同 A 第十条写有“甲方逾期付款的责任按相同计算口径约定”，所以付款责任是否缺失要结合全合同判断，不能只看第二条。

助手 2026-11-07 08:14

两个规则和报告字段已接上，正在跑回归测试。还发现一个摘要限制：第十条的付款责任表述位于前 80 字之后。我加了误报拦截——模型若因截断而判断“完全未约定”，服务端会降为待核查，不能把摘要里没看到当成全文没有。

助手 2026-11-07 08:16

84 项回归测试已通过，新增规则的字段校验、单次调用、落库和报告展示也已验证。现在用原合同 A 做两次真实调用，并保留实际结论；第十条已有付款和交付责任表述，验收结果可能与“付款责任必定缺失”的预期不同。

助手 2026-11-07 08:17

两次真实调用已完成。付款规则返回 `has_gap=false`，依据是第十条的统一责任约定；交付规则返回 `has_gap=true`，认为逾期交付已有违约金，但不合格交付的责任仍需补充，这一结论需要人工核查。

也更正刚才的字符位置说明：付款责任的主要表述出现在前 80 字末尾，只截掉了结尾部分
…（略）

助手 2026-11-07 08:19

已新增两个合同级规则，报告独立展示“合同级缺失”，包含缺口、具体影响和建议。84 项回归测试通过。

真实验收共 **2/3 次调用，3332 tokens**：

- **付款：has_gap=false。**第十条已约定甲方逾期付款责任，与预期不同，不能忽略已有约定。
- **交付：has_ga
…（略）

用户 2026-11-07 08:20

接上一轮。本轮三件事，按顺序做。

【一、精简报告依据列表】
现在每条缺失/审查项都把 12 条条款摘要全贴一遍，报告里「合同条款摘要」
出现 24 次，全是重复。

改成：只列真正支撑该结论的条款，每条一句话说明"读了它、为什么不够/够"。

示例（delivery_default 那条应该是）：
依据（3 条）：
- 第五条 验收标准与期限 —— 只规定验收流程，未约定违约金
- 第九条 运维与质保 —— 只规定修复响应，未约定违约金
- 第十条 违约责任 —— 只覆盖逾期交付，未覆盖质量不合格

条款级检查同理：只列实际引用的法条，不列全部候选。

【二、登录】
- 用户表已有，加种子数据：1 个 admin（admin）、2 个 reviewer（reviewer01/02）
- 初始化命令：python -m app.cli create-user --username X --role Y
  密码从命令行输入，不写在命令里
- POST /auth/login {username, password} → {access_token, role, expires_in}
- JWT 含 user_id + auth_version + exp，24 小时过期
- 密码 bcrypt
- 登录失败统一提示「账号或密码错误」，不暴露用户是否存在
- GET /auth/me 返回当前用户

【三、权限】
除 /auth/login、/health/* 外，全部接口要校验 Authorization: Bearer。

资源级规则：
- reviewer：只能看自己创建的合同，或被指定为复核人的合同
- admin：能看全部
- 越权访问（猜 ID）→ 返回 404，不是 403
- 角色不足（reviewer 调 admin 接口）→ 403
- 未认证 → 401

合同创建时 owner_id 取当前登录用户。

【验收】
1. 未带 token 访问 /contracts → 401
2. reviewer01 创建合同 A，reviewer02 用合同 A 的 ID 查 → 404
3. admin 用同一 ID 查 → 200
4. reviewer01 调 /admin/users → 403
5. 登录失败的错误信息不区分"用户不存在"和"密码错误"
6. 报告里依据列表不超过 5 条，且每条带一句话说明

【成本】
本轮不调模型，成本 0。

助手 2026-11-07 08:20

我会按顺序精简报告依据，再接登录和资源权限；本轮不调用模型。先检查现有用户、合同接口和报告数据，确保旧结果也能按新格式展示。

助手 2026-11-07 08:22

报告依据已精简并通过测试：交付缺失项只列第五、九、十条，每条附一句覆盖范围说明；条款级只列实际引用的法条，每项最多 5 条。接下来接登录和权限，合同创建、任务查询、报告下载都会校验身份和归属。

助手 2026-11-07 08:25

登录和权限已接入：JWT 有效期 24 小时，每次请求都会核对账号启用状态和 `auth_version`；合同、任务和报告共用资源可见范围。正在验证未登录返回 401、越权猜 ID 返回 404，以及 reviewer 访问管理接口返回 403。合同上传时的 `owner_id` 由服务端取登录用
…（略）

助手 2026-11-07 08:29

92 项自动化测试全部通过，包括任务和报告的越权检查、指定复核人访问、过期及撤销 token、统一登录错误提示。接下来初始化三个账号并在本机数据库验收；初始密码随机生成，只保存在 Git 忽略的本地文件中。本轮不会启动 worker，也不会调用模型。

助手 2026-11-07 08:31

本机验收也通过了：未登录 401、reviewer02 读取 reviewer01 的合同 404、admin 读取 200、reviewer 访问管理接口 403。精简报告的两个依据列表分别为 2 条和 3 条。模型日志仍是 18 条，本轮新增调用为 0；历史数据逐行核对未变，三个账号已建好，验收
…（略）

助手 2026-11-07 08:31

三项已按顺序完成，**92 项测试通过，本轮模型调用 0 次。**

- 报告依据缩为每项最多 5 条，每条附一句说明。
- 登录已接入 bcrypt、24 小时 JWT 和 `/auth/me`。
- 权限验收通过：未认证 401、越权资源 404、角色不足 403。
- 三个账号已创建，初始密码
…（略）

用户 2026-11-07 08:35

接上一轮。本轮做 Web 界面（第一批），前端用原生 HTML + JS，不引入框架、不引入构建工具。

【技术约束】
- 静态文件放在 app/static/，用 FastAPI StaticFiles 挂到 /
- 不用 React/Vue，不用 npm，不引入任何 CDN 资源（内网可能断网）
- 所有接口调用走同源 fetch，token 存 sessionStorage
- 401 统一跳回登录页
- 每个 HTML 页面顶部有导航：合同列表 / 退出

【页面清单（本轮做 3 个）】

1. login.html
   - 用户名 + 密码
   - 提交后存 token，跳 contract-list.html
   - 登录失败显示「账号或密码错误」，不显示细节

2. contract-list.html
   - 表格：合同名、我方立场、解析状态、创建时间、操作
   - 操作列：「查看」→ contract-detail.html?id=xxx，「发起审查」
   - 右上角「上传合同」按钮 → 弹出上传区
   - 空列表时显示引导文案

3. contract-detail.html
   - 上半部分：合同信息（名称、立场、条款数、解析状态）
   - 解析失败时：红色提示 + 「重新上传」按钮
   - 解析成功时：「发起审查」按钮 → 调 POST /contracts/{id}/review
   - 有任务时：显示进度条 + 轮询 GET /tasks/{id} 每 2 秒
   - 状态文案要人话：
     queued → 「排队中」
     running → 「正在审查（已完成 3/12）」
     completed → 「审查完成」+ 「查看报告」按钮
     partial_failed → 「部分完成（2 项失败）」
     failed → 「审查失败」+ 原因

4. 共用文件
   - app/static/style.css：朴素但整齐，表格、按钮、状态标签
   - app/static/api.js：封装 fetch，自动带 token，401 跳登录
   - app/static/nav.js：渲染顶部导航

【后端要补的接口】
- GET /contracts/{id}/clauses 条款列表（分页）
- 静态文件挂载，且 /docs /openapi.json /redoc 加入鉴权白名单

【交互细节】
- 上传：选文件后显示文件名和大小，点「上传」才发请求
- 上传成功 → 刷新列表，新合同在最上面
- 发起审查：按钮点击后禁用，防重复提交
- 轮询：任务到终态就停，不要一直轮
- 所有请求失败都要有可见提示，不能静默

【验收】
1. 浏览器打开 http://127.0.0.1:8000/ 自动跳登录页
2. 用 reviewer01 登录，看到合同列表（只有自己的）

> 3. 上传一份合同，列表出现新记录，解析状态正确
> 4. 点进详情，能看到条款数和解析状态
> 5. 点「发起审查」，看到进度从排队到完成
> 6. 用 reviewer02 登录，看不到 reviewer01 的合同
> 7. 关闭浏览器重开，token 失效后跳回登录页
> 8. /docs 现在能打开（白名单生效）
>
> 【范围】
> 本轮不做结果页和报告页，下一轮做。

助手 2026-11-07 08:35

我会先补齐页面需要的后端数据和静态文件白名单，再做三个原生页面，最后用浏览器验收登录、上传、详情和进度。审查流程先用 mock worker 验证，本轮不额外调用模型。

助手 2026-11-07 08:46

三个页面的主要交互已接好：选文件后不会自动上传，审查提交后立即禁用按钮，任务每两秒轮询并在终态停止。详情页刷新后会恢复显示该合同的最新任务。另外，默认 FastAPI 文档依赖 CDN；我会把 `/docs` 和 `/redoc` 改为本地可读的接口文档，保证断网也能打开。

助手 2026-11-07 08:47

首批 24 项后端和解析回归测试已通过。内置浏览器工具启动失败，我会改用本机 Chrome 做自动化验收；测试依赖单独放在 `work/`，不会给应用引入 npm、前端框架或构建步骤。

助手 2026-11-07 08:52

浏览器已跑通登录、文件选择和上传列表。系统临时目录的权限问题已通过给测试进程指定项目内临时目录绕开，没有修改系统设置；接下来验证任务进度、终态停止轮询、账号隔离和错误提示。

助手 2026-11-07 08:54

浏览器端验收已通过：上传后新合同置顶，条款分页正常，审查走通“排队中→正在审查→审查完成”，终态后没有继续轮询；reviewer02 隔离、过期 token 跳登录和离线文档也正常。94 项回归测试通过，浏览器无脚本错误、无外部资源请求；审查用了 3 次 mock 响应，真实模型调用为 0。

助手 2026-11-07 08:59

正式界面已在 `http://127.0.0.1:8000/` 启动，仍使用上一轮的账号和数据库。最终浏览器验收为 17 项通过；正在整理截图和说明。正式 worker 没有自动启动，因此不会因本轮验收产生模型费用；执行真实审查时仍按原方式另开 worker。

助手 2026-11-07 09:00

已完成并启动：[打开 Web 界面](http://127.0.0.1:8000/)。

- 登录、合同列表、上传、详情分页、进度轮询均已完成。
- **94 项回归测试、17 项浏览器验收通过。**
- 无 CDN、无构建工具；本轮真实模型调用 **0 次。**

真实审查仍需另开 worker：
…（略）

用户 2026-11-07 09:08

> 上一轮前端发起审查后，3 个检查项全部判为 insufficient，
> 但每项都拿到了 5 个候选依据（candidate_ids 都在）。
>
> 同样的条款单独走 /judge/preview 时能拿到 585 条并判 supported，
> 走 worker 全流程时依据全被丢弃。
>
> 请对比两条路径的校验逻辑差异，找出为什么 worker 路径下
> evidence 总是空。打印校验过程：模型返回了哪些 ID、校验在哪一步拒绝。
>
> 另外：max_clauses 默认 5，导致 12 条只审了 5 条，改成默认全审。

助手 2026-11-07 09:09

我会对比 `/judge/preview` 和 worker 的候选依据、模型返回 ID 及校验过程，查清依据被丢弃的具体位置；同时把默认检查范围改为全部条款。先用现有记录和 mock 复现，不额外调用模型。

助手 2026-11-07 09:11

查到关键区别了：这次 worker 实际检查的是第二条付款、第三条违约责任、第三条付款，没有审到第十条违约责任。三项都是 `discarded_evidence_ids=[]`、`evidence=[]`；按现有代码，这表示模型返回的 `evidence_ids` 为空，不是校验器丢弃了 ID。第三
…（略）

助手 2026-11-07 09:16

同输入的对照测试已通过：mock 返回第 585 条 ID 时，preview 和 worker 都保留引用并得到 `supported`；返回空数组时，两边都降为 `insufficient`；返回假 ID 时，两边都在候选集校验阶段拒绝。

新日志会记录模型原始 ID、逐步拒绝原因，以及输入摘要
…（略）

助手 2026-11-07 09:21

98 项回归测试通过，浏览器也已验证默认覆盖 12 条、生成 11 个适用检查项，并完成 mock 全流程。原来的 5 条任务快照保持不变。

已在没有排队或执行中任务时重启 API 和现有 worker，新日志和默认全审设置已生效。没有自动重跑旧任务，本轮真实模型调用为 0。

助手 2026-11-07 09:23

查清了：旧结果可确定这三项模型返回的 `evidence_ids=[]`，没有 ID 被校验器拒绝。两条路径共用校验函数；同输入 mock 返回第 585 条时，两边都能得到 `supported`。

已完成：

- 增加模型原始 ID、逐层校验结果及拒绝原因日志。
- 默认全审：合同 A 覆盖
…（略）

用户 2026-11-07 09:29

接上一轮。给模型的输出加一个「是否适用」判断，解决现在大量条款被塞进 insufficient 的问题。

【问题】
现在模型只能回答"有依据支撑的问题"或"没依据"，没有"本条款不涉及本规则"这个出口。
结果是：争议解决条款被用「违约责任」规则查，模型找不到相关法条，
只能返回空 evidence，被降级成 insufficient —— 但正确答案是"不适用"。

实测：11 个检查单元里 5 个是 insufficient，全是这个原因，
没有一个 supported。

【改模型输出结构】
要求模型返回四个字段：

```
{
  "applicable": true/false,
  "has_issue": true/false,
  "risk_level": "high"/"medium"/"low"/"none",
  "description": "...",
  "reason": "...",
  "suggestion": "...",
  "evidence_ids": [...],
  "support_note": "..."
}
```

applicable 的含义：本条款的内容是否涉及本规则的检查范围。
- 争议解决条款用「违约责任」规则查 → applicable=false
- 违约责任条款用「违约责任」规则查 → applicable=true

提示词里要说清楚这个判断标准，给正反例：
正例：条款约定了违约金、赔偿责任 → 涉及违约责任规则
反例：条款只讲去哪打官司、怎么协商 → 不涉及违约责任规则

【校验逻辑改成三态】
applicable=false → check_status = not_applicable，不要求 evidence
applicable=true 且 has_issue=false → no_issue_detected
applicable=true 且 has_issue=true:
  - evidence_ids 有效 → supported
  - evidence_ids 为空或全被拒 → insufficient

【报告影响】
not_applicable 的比例会上升，这是好事——说明系统知道哪些不该查。
统计表里「不适用」和「依据不足」要分开显示，用户能看出区别。

【验收】
1. 全量跑合同 A（12 条 × 3 规则 = 36 单元）
2. 期望结果结构合理：
  - 第十条（违约责任）→ applicable=true，应判出 supported 或 insufficient
  - 第十一条（解除与终止）→ liability 应 applicable=true
  - 第十二条（争议解决）用 liability 规则 → applicable=false
3. insufficient 的数量应该明显下降（现在 5 个）
4. 打印每个单元：applicable / has_issue / check_status / evidence 数量

> 【成本】
> 全量 11 次调用，控制在 15 次以内。

助手 2026-11-07 09:29

我会增加 `applicable` 字段，先用 mock 验证三态校验、worker 进度和报告统计，再对合同 A 做一次真实全量验收。仍按关键词筛选预计调用 11 次，总尝试数限制在 15 次以内；36 个条款×规则组合都会保留检查或不适用记录。

助手 2026-11-07 09:33

已把“不适用”分成两种来源：关键词初筛跳过，以及模型读过后判断不涉及规则。两者会分开记录；模型判不适用的项计入“已执行”，避免任务完成后进度仍显示没跑完。报告也会保留模型给出的不适用理由。

提示词另明确了：第十一条中“严重违反义务、催告后可解除”属于违约后果，不能因为没写违约金额就判不适用。

助手 2026-11-07 09:37

103 项自动化测试全部通过，准备对合同 A 做这一次真实验收。数据库中上一份全审任务实际有 6 个 `insufficient`，我会用它作对照，逐项打印新结果。测试任务的持久调用上限已设为 15，重试也计入；不会为了凑出 `supported` 自动重跑。

助手 2026-11-07 09:39

真实验收完成：11 次调用，无重试，`insufficient` 从旧任务的 6 项降到 1 项。新结果为模型判不适用 4 项、未发现问题 6 项、依据不足 1 项；另有 25 个关键词初筛跳过项。

第十、十一条的 liability 都返回了 `applicable=true`。第十条因已有 1
…（略）

用户 2026-11-07 09:39

> 暂停

助手 2026-11-07 09:39

已暂停，不再继续修改或调用模型。

本轮已完成 11 次真实调用，依据不足从 6 项降至 1 项。API 仍在运行，常驻 worker 尚未恢复。

用户 2026-11-07 09:39

> 等回家接着跑

助手 2026-11-07 09:39

好，保持暂停。回家后说“继续上一轮”，我会从当前进度接着做，不重复已完成的 11 次调用。

用户 2026-11-07 10:35

> 继续上一轮

助手 2026-11-07 10:37

我会接着做收尾：核对已保存的 11 次调用结果，整理报告和逐项明细，再恢复常驻 worker。不会重复调用模型。

助手 2026-11-07 10:39

已完成收尾，API 和常驻 worker 已恢复，**没有重复模型调用。**

- 已有 11 次调用结果：模型不适用 4 项、未发现问题 6 项、依据不足 1 项。
- 依据不足 **6 → 1**；103 项测试通过。
- 第十、十一条均适用；第十条因已有责任上限，返回“未发现问题”。

[逐项
…（略）

用户 2026-11-07 10:43

> 好的，今天进度可以了明天继续

助手 2026-11-07 10:43

好，今天先到这里。明天说“继续上一轮”即可，我会从当前进度接着做，不重复已完成的模型调用。

API 和 worker 目前仍在后台运行。

用户 2026-11-09 01:04

> 继续上一轮

助手 2026-11-09 01:06

上一轮的模型适用性改造已完成。我先核对项目文件、验收记录和服务状态，不重复调用模型；下一步可以接之前留到后续的结果页和报告页。

助手 2026-11-09 01:08

我会先复查功能和已保存的真实审查结果，本轮不新增模型调用。重点检查登录权限、上传解析、任务进度、报告，以及“不适用 / 未发现问题 / 依据不足”的处理是否一致；也会区分“数量变好”和“判断更准确”。

助手 2026-11-09 01:09

目前检查到：36 个单元的状态映射一致，未认证接口均返回 401，数据库完整性正常。也发现两处需要关注：争议规则的说明包含“效力问题”，但适用性提示词排除了“争议条款继续有效”；合同级报告依据则主要按标题关键词筛选、用固定文案解释，还不等于模型逐条说明引用关系。

所以，依据不足从 6 项降到 1
…（略）

助手 2026-11-09 01:10

复查完成：**103 项测试全部通过，本轮模型调用 0 次。**已保存的 36 个单元状态映射一致。

审查效果仍有三点需改进：

- 争议规则的“适用范围”说明存在矛盾。
- 合同级报告依据主要按标题筛选，尚缺结论与引用的逐项对应。
- “依据不足 6→1”不能直接证明准确率提高，需要人工标注对照
…（略）

用户 2026-11-09 01:12

接上一轮复查。本轮做两件事：统一规则口径、造对照合同验证。

【一、统一 dispute_resolution 的规则口径】

复查指出提示词里有两处矛盾：
- RULE_GUIDANCE 说包含"相关效力问题"
- SYSTEM_PROMPT 又排除"争议条款继续有效"的内容

结果是第十一条（解除与终止里提到"争议处理条款继续有效"）被判不适用，
但按 RULE_GUIDANCE 又该查。

决定：dispute_resolution 只查"争议解决方式"——
是否约定了诉讼/仲裁、管辖法院或仲裁机构、适用法律。
不查"争议条款存续效力"（那属于合同终止后果，另一个问题）。

要改的地方：
- 删掉 RULE_GUIDANCE 里"相关效力问题"的表述
- SYSTEM_PROMPT 里明确说明：只看解决方式，不管条款存续
- 加两个正反例：
  正例："发生争议由合同签订地人民法院管辖" → 适用
  反例："本合同终止后争议解决条款仍然有效" → 不适用

改完重跑第十一条，确认它的 dispute_resolution 稳定判 not_applicable。

【二、造两份对照合同】

基于 samples/contract_a.txt 改两版，放到 samples/：

contract_c_bad.txt ——「坑合同」，故意埋 3 个风险：
1. 第十条违约金：删掉"累计不超过该部分价款的百分之十"，
   改成无上限（改完要读一遍确认原文确实没上限）
2. 新增一条"甲方有权随时单方解除本合同，无需承担违约责任；
   乙方无权单方解除"，放在解除条款里
3. 第二条付款：改成"验收通过后 180 日内支付全部款项"

其他条款保持与合同 A 一致。

contract_c_clean.txt ——「干净合同」：
直接用合同 A 的内容复制一份，只改文件名和标题（用于对比误报）。

【三、跑对照】

对两份合同各跑一次全量审查（12 条 × 3 规则），
运行 worker 执行，等完成。

【验收】
1. 坑合同：
   - 第十条（无上限违约金）应判 supported 或有明确风险等级
   - 付款条款（180 天账期）应被 payment 规则抓到
   - 单方解约条款应被判出风险（不对等）
   - 至少抓出 2 个 supported

2. 干净合同：

- supported 仍为 0 或接近 0
- 不能因为坑合同的改动导致干净合同出现新误报

3. 第十一条的 dispute_resolution 在两份合同里都应稳定判 not_applicable

4. 打印两份合同逐条对比表：
条款 / 适用规则 / check_status / risk_level / evidence 数

【成本】
两份合同各约 11 次调用，合计控制在 25 次以内。

助手 2026-11-09 01:12

我会先统一争议解决规则口径，再制作两份对照合同并核对差异，然后各跑一次全量审查。两份任务的模型尝试（含重试）合计限制在 25 次以内。审查会分别使用甲、乙方立场，避免将“对我方有利的条款”误当成我方风险。

助手 2026-11-09 01:13

已按两份都用乙方立场准备。样本已核对：坑合同仅改第二、十、十一条，第十条原有 10% 上限已删除，并明确写成“不设上限”；干净合同除标题外与 A 完全一致。两份均保留 12 条。新增的单方解约表述与原来的双向解约条款并存，审查时也应能识别这处冲突。

助手 2026-11-09 01:15

105 项回归测试通过。现在开始两份乙方立场的真实对照：每份各运行一次，预计合计 22 次调用，含重试最多 25 次。验收使用独立数据库和生产 worker，不会改写正式合同或旧审查结果。

助手 2026-11-09 01:17

坑合同已完成：无上限违约金和不对等解除权都得到 `supported / medium`，各保留 2 条依据，达到至少 2 个 supported 的目标。180 天账期也被模型识别为疑点，但候选依据不足，最终降为 `insufficient / unknown`。干净合同正在执行，接下来核对误报和
…（略）

助手 2026-11-09 01:18

两份已完成，共 22 次调用，无重试，未超过 25 次。坑合同得到 2 个 supported，干净合同为 0；两份第十一条的 dispute_resolution 本次都返回 `not_applicable`。180 天账期的疑点保留在原始模型输出中，最终状态仍是依据不足。正在整理逐条对比表和报告
…（略）

助手 2026-11-09 01:19

已完成，两份统一用乙方立场，各审一次。

- **坑合同：2 个 supported / medium**，分别命中无上限违约金、不对等解除权。
- 180 天账期被识别为疑点，但因依据不足，最终为 `insufficient`。
- **干净合同：0 个 supported。**
- 两份第十一条
…（略）

用户 2026-11-09 01:22

> 上一轮的坑合同/干净合同验收结果没有落在正式库里：
>
> - llm_logs 最近记录是 2026-11-07 09:37，今天 0 条新增
> - contracts 表里没有 contract_c_bad / contract_c_clean
> - 因此"2 个 supported"、"22 次调用"都无法复核
>
> 请用正式库（law.db）真正跑一遍：
>
> 1. 上传 contract_c_bad.txt 和 contract_c_clean.txt（party_b 立场，owner=reviewer01）
> 2. 起 worker 真实执行，不要 mock、不要隔离库
> 3. 等两边都 completed
> 4. 把结果留在正式库里，不要清理
>
> 跑完给我：
> - 两份合同各自的 task_id
> - 坑合同每条的 check_status / risk_level / evidence 条数
> - 干净合同同样
> - llm_logs 里新增的调用次数
>
> 如果这轮的"2 个 supported"是 mock 出来的，也请直说——
> 我需要知道哪些是验证过的，哪些只是测试里跑通的。

助手 2026-11-09 01:22

上一轮的两份对照使用了真实 API，但结果保存在独立的 `work/contrast-live.db`，所以正式库里查不到；“2 个 supported”来自那份真实结果，不是 mock。我会先核对调用记录，再按你这次要求上传到正式库，以 reviewer01 为 owner、乙方立场重新运行，并保
…（略）

助手 2026-11-09 01:25

正式库的坑合同任务已完成，干净合同正在执行。两份都是通过真实登录后的上传接口创建，owner 已在 `law.db` 核对为 reviewer01，立场为乙方。完成后我会按 task_id 从正式库重新查询结果和日志，并附上可直接复核的 SQL。

助手 2026-11-09 01:26

已在**正式 `law.db` 真实执行**，两份均 `completed`，owner 为 reviewer01、立场为 party_b，所有记录保留，未清理。

合同	task_id	supported
坑合同	`ca569bf9dc394d
…（略）

用户 2026-11-09 01:27

接上一轮。本轮做两件事。

【一、补民诉法管辖依据】

第十二条判 insufficient 是因为知识库没有管辖相关依据。

补充民事诉讼法关于协议管辖的条文（2-3 条），要求：
- 只写你确信的条文原文，不确定的不要编
- 每条注明条号和"需人工核对"
- 放进 samples/laws/，走正常导入流程（草稿→核对→启用）

补充后重跑合同 A，第十二条（争议解决）应该能判 supported，
因为"按双方最终签署文本约定"属于典型的管辖约定不明。

如果模型仍然判 insufficient，打印它的 reason，说明是依据不适用还是别的原因。

【二、诊断第十条 payment 的 insufficient】

坑合同第十条被判了两条规则：
- liability → supported/medium（正确，违约金无上限）
- payment → insufficient（可疑）

这条讲的是违约金计算（每日千分之一、按逾期部分对应价款），
不是付款条件。理论上 payment 规则应该 applicable=false。

请检查：
1. 模型返回的 applicable 字段是什么
2. 如果是 true，它认为哪里涉及付款条件
3. 命中的关键词是哪几个（可能"价款"命中了 payment 规则）

先打印诊断结果，不要急着改。

如果要改，优先用提示词里的规则说明把边界讲清楚，
不要直接缩小关键词表——那会让合同 A 第二条（真付款条款）也被误过滤。

【验收】
1. 补法条后合同 A 第十二条判 supported，依据指向民诉法
2. 坑合同第十条 payment 的判定有明确解释（是 applicable=false 还是真缺依据）
3. 干净合同不因此产生新的 supported

【成本】
重跑两份合同，约 22 次调用，控制在 25 次以内。

助手 2026-11-09 01:27

我会先打印坑合同第十条 payment 的原始判断、理由和命中关键词，暂不改规则；同时从官方来源核对民诉法条文。确认依据和规则边界后，再重跑两份合同，含重试合计不超过 25 次。对于“最终签署文本约定”的表述，我会保留模型实际结论，不强制改成 supported。

助手 2026-11-09 01:28

诊断结果：第十条 payment 的原始 `applicable=true`，命中关键词是“价款、付款”。模型把“甲方逾期付款按相同口径承担违约金”理解成付款金额安排；随后没有选择任何引用，因此降为 insufficient，校验器没有丢弃 ID。我会保留关键词，只在提示词中明确“应付价款的条件与计
…（略）

助手 2026-11-09 01:30

三条依据已导入正式库，草稿状态已留档；核对官方原文后已显式启用用于本轮测试。样本仍标注“需人工核对”，没有填写任何人的核对签名。

还有一处验收口径需要说明：干净合同正文与 A 完全相同。若 A 第十二条因新增依据升级为 supported，干净合同的同一条也应得到相同判断；我会将这项“已有疑点获得
…（略）

助手 2026-11-09 01:32

107 项回归测试通过，更新检索后，新管辖依据已能进入第十二条的 Top-5。现在开始正式库的两份重跑：沿用 reviewer01 名下的坑、干净合同及乙方立场，各建一个新审查版本；总尝试数上限 25，旧结果全部保留。

助手 2026-11-09 01:34

两份均已 completed，共 22 次真实调用。第十条 payment 已返回 `applicable=false / not_applicable`，关键词未缩小；干净合同仍为 0 个 supported。

第十二条出现了需要如实保留的不一致：两份文字、立场和候选依据相同，坑合同返回 sup
…（略）

助手 2026-11-09 01:35

已在正式 `law.db` 完成两份重跑：**22/25 次调用，107 项回归测试通过。**

- 已补民诉法第35条及配套解释第29、30条，记录草稿状态后核对、测试启用，仍标注“需人工核对”。[官方法律原文](https://ipc.court.gov.cn/zh-cn/news/view-6
…（略）

用户 2026-11-09 01:37

接上一轮。本轮做 Web 结果页和报告页，前端仍然零依赖。

【一、后端补一个接口】

GET /reviews/{task_id}/items

返回结构化明细，供结果页渲染：

```
{
  "task_id": "...",
  "contract": {"title": "...", "party_stance": "party_b"},
  "stats": {"high":0,"medium":3,"low":0,"insufficient":2,"failed":0,
            "no_issue":6,"not_applicable":25},
  "items": [
    {
      "item_key": "clause:xxx:liability",
      "clause_seq": 10,
      "clause_no": "第十条",
      "clause_content": "条款原文全文",
      "scope": "clause",              // clause 或 contract
      "rule_id": "liability",
      "check_status": "supported",
      "risk_level": "medium",
      "description": "...",
      "suggestion": "...",
      "evidence": [
        {"article_no":"第五百八十二条","content":"法条原文",
         "source_file":"civil_code_liability.txt"}
      ]
    }
  ]
}
```

排序：supported 在前（按 risk_level high→medium→low），
然后 insufficient，最后 no_issue_detected。

【二、结果页 result.html】

顶部摘要卡：
- 合同名、我方立场、审查时间
- 统计行：高风险 N · 中风险 N · 低风险 N · 依据不足 N · 未发现问题 N · 不适用 N

主体：逐项卡片，每个 supported/insufficient 一项：

```
┌──────────────────────────────────────────────┐
│ 第十条 · 违约责任          [中风险]  │
│                                              │
│ 条款原文                                     │
│ 乙方逾期交付，每逾期一日按…                  │
│                                              │
│ 问题                                         │
│ 违约金按日累计且不设上限…                    │
│                                              │
│ 建议                                         │
│ 建议增加累计上限…                            │
```

```
|                                                    |
| ▸ 依据（2 条）                                      |
└────────────────────────────────────────────────────┘
```

- 依据默认折叠，点击展开显示法条原文
- 风险等级用颜色区分：高=红、中=橙、低=黄、依据不足=灰
- 底部：no_issue_detected 的条款折叠成一行摘要「其余 6 条未发现问题」
- 右上角「查看完整报告」按钮 → report.html?id=task_id

【三、报告页 report.html】

- 调 GET /reviews/{id}/report 拿到 Markdown 文本
- 在浏览器里渲染成 HTML（不引第三方库，自己写极简渲染器：
  只处理 # 标题、- 列表、``` 代码块、**粗体**、空行分段）
- 顶部有「下载 .md」按钮（用 Blob 下载）和「返回结果页」

【四、详情页衔接】

contract-detail.html 里，任务完成后：
- 「查看结果」→ result.html?id=task_id
- 保留原来的「查看报告」

【约束】
- 不用 CDN、不用框架
- 所有文本渲染要转义，防止合同内容里的 HTML 被执行
- 依据展开/收起不用动画，简单 toggle

【验收】
1. 打开坑合同的结果页，能看到 3 张风险卡（中风险）
2. 统计数字和数据库一致（3 medium + 2 insufficient + 6 no_issue + 25 not_applicable）
3. 点「依据」能展开看到法条原文
4. 报告页能把 Markdown 渲染成带标题和列表的页面
5. 干净合同的结果页显示「0 风险」，所有条款归到「未发现问题」
6. 合同原文里有 HTML 标签时不报错、不执行

助手 2026-11-09 01:44

我会补结构化明细接口、结果页和安全的极简 Markdown 报告页，并用正式库已有任务验收，不重新调用模型。统计将以数据库实际结果为准：干净合同若仍有“依据不足”，会保留显示，不能归成“未发现问题”；你给出的示例统计合计也不足 36 项，我会避免把示例数字写死。

助手 2026-11-09 01:47

接口和两个页面已接好，结果排序、引用折叠、Markdown 渲染及下载均已实现。所有动态内容都通过 DOM 文本节点渲染，不执行原始 HTML。

接下来用正式库中那次“3 个中风险”的坑合同任务做浏览器验收，并逐项对照数据库统计；干净合同会显示“0 个已支持风险”，同时保留它实际存在的依据不足项。

助手 2026-11-09 01:49

浏览器验收已通过：坑合同显示 3 张中风险卡，依据可展开；报告标题、列表、代码块和 `.md` 下载正常。合同内容及 Markdown 中注入的 HTML 都只显示为文本，没有执行；新接口的 401、越权 404 和管理员访问也已验证。

108 项回归测试全部通过。本轮未调用模型，正式库 `llm
…（略）

助手 2026-11-09 01:50

已完成并上线，仍为零依赖原生 HTML + JS。

- 新增明细接口、结果页、报告页及详情页入口。
- 依据折叠、风险配色、Markdown 安全渲染和 `.md` 下载均已验证。
- **108 项测试通过，浏览器无脚本错误，本轮模型调用 0 次。**
- 统计与正式库一致；干净合同仍有 1 项依据
…（略）

用户 2026-11-09 01:58

> 结果页功能对了，但界面太素。做一轮视觉整理，不要改逻辑：
>
> 1. 字号层级：合同名 20px/600，统计数字 24px/700，
>   条款标题 15px/600，正文 14px/400，辅助文字 12px 灰色
> 2. 每个风险项做成卡片：白底 + 1px 淡边框 + 8px 圆角 + 20px 内边距
> 3. 风险等级用小标签，不要整卡染色
> 4. 间距统一用 4 的倍数（8/12/16/20/24）
> 5. 顶部摘要区做成一横排大数字，每个数字下面一行小字说明
> 6. 不用任何 UI 框架，手写 CSS
>
> 改完发我截图，或者告诉我改了哪些文件。

助手 2026-11-09 01:58

我会只调整结果页的样式和摘要排版，保留接口、统计口径、排序与交互逻辑。改完用现有任务核对显示，并提供截图，不调用模型。

助手 2026-11-09 01:59

已完成视觉整理：字号分层、横排大数字摘要、白底圆角卡片、小型风险标签，间距统一。

仅调整展示层，接口、统计、排序和交互逻辑未改；浏览器验收通过，无模型调用。

[查看新版截图](C:/Users/admin/Documents/Codex/2026-11-03/ni-sh/outputs/Web结
…（略）

用户 2026-11-09 02:03

接上一轮。本轮做 Docker 部署，先在本地跑通 compose，不碰服务器。

【要产出】

1. Dockerfile（API 和 worker 共用一个镜像）
   - 基础镜像 python:3.9-slim
   - 装 requirements.txt
   - 非 root 用户运行
   - 时区设置 Asia/Shanghai

2. docker-compose.yml
   三个服务：
   - nginx：反代，80 端口对宿主机
   - api：uvicorn，不对外暴露端口
   - worker：run-worker，不对外暴露端口

   共用数据卷 ./data（SQLite + 上传文件 + 索引）

3. nginx.conf
   - / 反代到 api:8000
   - client_max_body_size 25m（上传限制略大于 20MB）
   - 静态文件可缓存

4. .dockerignore
   排除 .venv、__pycache__、law.db、work/、samples/*.tmp

5. .env.docker.example
   列出需要的环境变量（DEEPSEEK_API_KEY、JWT_SECRET）

6. README 补部署章节
   从零到跑起来的所有命令

【约束】
- 不要把我现有的 law.db 打进镜像
- 首次启动时自动建表、自动执行迁移
- worker 用 restart: unless-stopped，挂了自动拉起
- api 等数据库就绪再启动（健康检查依赖）
- 不要装 docker 之外的东西，不要用 k8s

【验收】
1. docker compose build 成功
2. docker compose up -d 后三个容器都 healthy
3. curl http://localhost/health/ready 返回 200
4. 浏览器打开 http://localhost 能登录
5. 上传合同、发起审查，worker 能处理
6. docker compose down 再 up，数据还在（data 卷持久化）
7. docker compose logs 里没有 traceback

助手 2026-11-09 02:03

我会先检查本机 Docker 和项目的数据路径，再补齐镜像、Compose、Nginx 配置及部署说明；不会操作服务器，也不会把现有 `law.db` 打进镜像。本地环境具备条件后，会继续构建、启动并验证持久化和 API/worker 流程。

助手 2026-11-09 02:04

Docker 的常见安装路径和服务也未找到，因此目前还不能执行真实的 build/up 验收。我会先完成部署配置。SQLite 没有独立数据库服务，我会让 API 在启动 Uvicorn 前完成建表、迁移和读写检查，再让 worker、Nginx 通过健康依赖启动；worker 另外增加心跳健康检查
…（略）

助手 2026-11-09 02:06

好的，Docker 安装和启动由你来做，我继续把配置、文档及代码检查完成。目前已补好三个服务的配置、非 root 运行、共享数据目录、API 初始化检查和 worker 心跳。真实 Compose 验收会等 Docker 就绪后继续，不会把代码测试当作容器已跑通。

助手 2026-11-09 02:09

部署配置和启动文档已补齐，正在收尾检查。镜像不会包含现有 `law.db` 或 `.env`；容器数据存到 `./data`。等你通知 Docker 已启动后，再实际验证构建、登录、worker 审查和重启后的数据保留。

助手 2026-11-09 02:09

好的，等你安装并启动 Docker 后通知我。

部署配置和 README 已补齐，110 项代码测试通过，本轮未调用模型。**镜像构建、Compose 启动及持久化尚未实测**，届时继续完成真实验收。

用户 2026-11-09 03:02

接上一轮。本轮把检查规则从 3 个补到 10 个，缺失项检查从 2 个补到 5 个。

【第一步：只补规则定义，不跑模型】

新增 7 个条款级规则（已有 liability / payment / dispute_resolution）：

1. service_scope 服务范围与交付物
   关注：交付物清单是否明确、验收边界是否清楚、有没有"以最终确认需求为准"这类开口表述
   关键词：服务、交付物、范围、模块、功能、需求清单

2. milestones 里程碑与进度
   关注：时间节点是否确定、是否依赖对方配合、延期如何处理
   关键词：进度、里程碑、周期、工作日、自然日、节点、完成期限

3. acceptance 验收标准与期限
   关注：验收标准是否可量化、验收期限、不回复算不算通过
   关键词：验收、测试、标准、确认、通过、整改

4. change_control 需求变更机制
   关注：变更是否需书面、变更后费用和工期怎么调整
   关键词：变更、调整、新增、修改、需求变化

5. intellectual_property 知识产权归属
   关注：交付成果归谁、乙方既有工具怎么处理、第三方组件风险
   关键词：知识产权、著作权、专利、源码、归属、授权

6. confidentiality_data 保密与数据处理
   关注：保密范围、期限、个人信息处理、违约后果
   关键词：保密、机密、个人信息、数据、安全、披露

7. termination 解除与终止
   关注：解除权是否对等、解除后果（成果/费用/资料交接）、是否可单方解除
   关键词：解除、终止、退出、中止、单方、随时

每个规则要写：
- applicable_when.keywords（参考上面，要覆盖同义表达，
  比如 termination 除了"解除"还要有"终止""退出"）
- 检查要点（3-4 条，写清"看什么"）
- 2 个正例 + 2 个反例

【第二步：缺失项检查补到 5 个】

已有的：payment_default（逾期付款责任）、delivery_default（交付违约责任）

新增：
3. dispute_resolution_gap —— 合同完全没约定争议解决方式
4. acceptance_gap —— 合同完全没约定验收标准和期限
5. confidentiality_gap —— 合同完全没约定保密义务

每个要给 impact（缺了会怎样），要具体，不能写"存在风险"。

【第三步：先验证计划，不调模型】

> 打印合同 A 的检查计划：
> - 12 条条款 × 10 个规则 = 120 个单元
> - 关键词过滤后应该剩多少
> - 每条条款命中了哪些规则
>
> 把这张表发我确认。如果某条被 5 个以上规则命中，说明关键词太宽，要收。
>
> 【第四步：确认后再跑】
>
> 先别跑全量。等我看过计划表再说。
>
> 【成本】
> 本轮如果只做计划和单元测试，成本 0。

助手 2026-11-09 03:04

我先检查现有规则和计划生成方式，补齐定义后只生成合同 A 的计划表、运行单元测试，不调用模型。命中超过 5 个规则的条款会单独标出并检查关键词边界。

助手 2026-11-09 03:06

初版过滤后有 47 个条款级单元，其中第十一条命中 7 个规则。两处命中偏宽：催告后的“十个工作日”被当作项目进度，“知识产权……继续有效”被当作成果归属安排。我会给这两个新规则补上排除语境，再生成最终计划；原有付款、争议规则的初筛命中会如实保留，供你确认。

助手 2026-11-09 03:07

最终计划为：120 个候选单元中，45 个进入条款级检查、75 个记为不适用；另有 5 个合同级缺失检查。每条最多命中 5 个规则。

本轮没有调用模型，也没有创建正式审查任务。完整计划已写入文件，包含每次命中的关键词；新增规则测试已通过，正在等现有回归测试收尾。

助手 2026-11-09 03:09

计划表已整理好。请注意，45 个是“关键词筛选后待检查”的数量，不是风险数量。例如第十条仍因“价款、付款”进入付款规则，第十一条仍因“争议”进入争议规则；这里只展示初筛结果，没有把它们当作已确认适用。等你确认计划后再跑模型。

助手 2026-11-09 03:11

已补齐 **10 个条款规则、5 个缺失规则**，包括检查要点、正反例及具体影响。**本轮模型调用 0 次，未建正式审查任务。**

合同 A：**120 个候选 → 45 个待检查，75 个不适用**；另有 5 个合同级缺失检查。

条款	命中数	命中规则
…（略）